

Supplementary Technical Material

Real-Time Shill-Bidding Detection via Online Bid-Graph Analytics in Live Auction Platforms

Asha Joseph Pratham Reet Nitin A. Rane Reeshav Sinha Om Ji Rao

*Department of Computer Science and Engineering, New Horizon College of Engineering,
Bengaluru, India*

Contents

1	Introduction	2
2	Simulator Architecture and Agent Specifications	2
2.1	Discrete-Event Simulation Environment	2
2.2	Agent Persona Specifications	2
2.2.1	Truthful Bidders (2 agents)	3
2.2.2	Sniper Bidder (1 agent)	3
2.2.3	Shill Bidder (1 agent)	3
2.2.4	Collusion Ring (2 agents)	3
2.3	Simulation Output Schema	3
3	Fraud Detection Engine – Complete Source Code	4
3.1	Type Definitions (fraud.types.ts)	4
3.2	Sliding-Window Bid Graph (fraud.graph.ts)	4
3.3	Feature Extraction (fraud.features.ts)	6
3.4	Logistic Regression Classifier (fraud.classifier.ts)	6
3.5	Engine Orchestrator (fraud.engine.ts)	8
4	Dataset Generation and Training Pipeline	9
4.1	Dataset Construction	9
4.2	Feature Scaling: Z-Score Normalisation	9
4.3	Gradient Descent Training	10
4.4	Learned Model Parameters	10
4.5	Training Convergence	10
5	Extended Experimental Results	11
5.1	Full ROC Sweep	11
5.2	Detailed Feature Ablation	12
5.3	Baseline Comparison	12
5.4	Throughput Benchmarks: Extended k6 Results	12

6	Formal Proofs and Complexity Analysis	13
6.1	Commit-Reveal Protocol: Security Properties	13
6.2	Graph Algorithm Complexity	13
6.3	Gradient Descent Convergence	14
7	k6 Load Test Harness	14
8	Sealed-Bid Commitment: Implementation	15
9	Reproducibility Instructions	17
9.1	Prerequisites	17
9.2	Setup and Execution	17
9.3	Output Artefacts	17

1 Introduction

This supplementary document provides the complete technical specifications, source code listings, dataset schemas, extended experimental results, and formal security proofs that support the primary manuscript.

Transparency note. Our research does not use any external or pre-existing dataset. Instead, we built a multi-agent discrete-event simulator (`packages/simulator/`) that drives real HTTP traffic against the live AuctionHaus backend. The simulator spawns four classes of behavioural agents (truthful, sniper, skill, and collusion), and every bid event it generates carries a ground-truth label (`isSkill`) assigned by the simulator at creation time – not post-hoc by human annotators. This design choice ensures that every label is deterministic, reproducible, and free from annotator bias. We present the simulator architecture, agent specifications, and the full training/evaluation pipeline in enough detail to enable complete, independent replication.

All source code referenced herein is publicly available at:

<https://github.com/prathamreet/auctionhaus>

The document is organised as follows: Section 2 details the simulator architecture and agent specifications; Section 3 presents the complete source code of the fraud detection engine; Section 4 documents the dataset generation pipeline and feature distributions; Section 5 provides extended experimental results including full ROC sweep data and per-threshold confusion matrices; Section 6 contains formal proofs for the commit-reveal protocol and graph-algorithm complexity bounds; Section 7 gives the k6 load-test harness source and full benchmark tables; Section 8 presents the sealed-bid commitment implementation; and Section 9 provides step-by-step reproducibility instructions.

2 Simulator Architecture and Agent Specifications

2.1 Discrete-Event Simulation Environment

The multi-agent simulator operates as a discrete-event loop that drives real HTTP traffic against the live AUCTIONHAUS backend. Each simulation run creates a fresh English auction, registers six synthetic bidder accounts, deposits wallet funds, and polls auction state at a configurable tick interval. Table 1 lists the simulation parameters.

Table 1: Simulation Environment Parameters

Parameter	Value
Auction Format	English (ascending)
Auction Duration	60 seconds
Starting Price (p_0)	INR 1,000
Minimum Increment (δ)	INR 100
Anti-Sniping Extension	Disabled (0 min)
Polling Interval (tick)	500 ms
Wallet Deposit per Agent	INR 500,000 ($5 \times 100,000$)
Backend Transport	HTTP REST (axios)

2.2 Agent Persona Specifications

Six agents are spawned per simulation run, belonging to four behavioural archetypes. Each agent implements a `decide(state, myUserId)` method that returns a `BidDecision` indicating whether to bid and at what amount.

2.2.1 Truthful Bidders (2 agents)

Each truthful agent draws a private valuation $v \sim \mathcal{U}(1.5 p_0, 3.5 p_0)$ and a random inter-bid interval $\Delta t \sim \mathcal{U}(2s, 20s)$. The agent bids when three conditions hold: (i) the current price is below v ; (ii) at least Δt has elapsed since the agent's last bid; and (iii) the agent is not already the top bidder. The bid increment is $\delta \times k$ where $k \sim \mathcal{U}\{1, 2, 3\}$. After each bid, a fresh Δt is sampled. Ground-truth label: `isShill = false`.

2.2.2 Sniper Bidder (1 agent)

The sniper draws a high valuation $v \sim \mathcal{U}(2 p_0, 5 p_0)$ and remains dormant until fewer than 15% of the auction duration remains. It then places exactly one bid at $\min(v, p_{\text{current}} + 5\delta)$. This models the well-documented last-minute bidding strategy. Ground-truth label: `isShill = false`.

2.2.3 Shill Bidder (1 agent)

The shill agent is parameterised with a price ceiling $c = 3 p_0 - 0.05 p_0$ (just below the seller's reserve). Its behavioural signals are:

- **Fast response:** delays only 200–800 ms after a competing bid (randomised uniformly).
- **Minimum increments:** always bids exactly $p_{\text{current}} + \delta$ (increment ratio $\rho = 1.0$).
- **Never wins:** stops when $p_{\text{current}} \geq c$.
- **Seller affinity:** targets all auctions by the same seller.

Ground-truth label: `isShill = true`.

2.2.4 Collusion Ring (2 agents)

Two coordinated accounts, A and B, alternately outbid each other. Agent A bids only when Agent B is the top bidder, and vice versa, creating a high reciprocity signal. Both use a shared ceiling $c \sim \mathcal{U}(3 p_0, 5 p_0)$ and response delays of 300–800 ms. The increment is $\delta \times k$ where $k \sim \mathcal{U}\{1, 2\}$. Ground-truth label: `isShill = true`.

2.3 Simulation Output Schema

Each run produces two files in `packages/simulator/runs/<runId>/`:

- `events.jsonl` – One JSON object per line per accepted bid. Fields: `ts` (Unix ms), `auctionId`, `bidderId`, `amount` (INR), `agentType` $\in \{\text{truthful}, \text{sniper}, \text{shill}, \text{collusion}\}$, and `isShill` (boolean ground truth).
- `manifest.json` – Run metadata: `runId`, `timestamps`, full `SimulationConfig`, `agentMap` (`userId` $\rightarrow$ `agentType`), and aggregate counts (`totalBids`, `shillBids`).

A sample `events.jsonl` record:

```
1 {"ts":1748700021543,"auctionId":"a1b2c3",
2  "bidderId":"u4d5e6","amount":1200,
3  "agentType":"shill","isShill":true}
```

2.4 Sample Simulator Output Log

The following is a verbatim excerpt from a real simulation run (`run_id: 353c37c5`), included here to demonstrate the ground-truth labelling and bid interleaving:

Listing 1: Actual simulator console output (abbreviated).

```
1 [Sim] Run 353c37c5 started -- duration 60s
2 [Sim] Auction 7ae6797f created
3 [Sim] truthful      bid 1,200 | price=1,000
4 [Sim] shill         bid 1,300 | price=1,200
5 [Sim] collusion     bid 1,400 | price=1,200
6 [Sim] shill         bid 1,500 | price=1,400
7 [Sim] collusion     bid 1,600 | price=1,400
8 [Sim] shill         bid 1,700 | price=1,600
9 [Sim] collusion     bid 1,800 | price=1,600
10 [Sim] truthful     bid 2,100 | price=1,800
11 [Sim] shill        bid 2,200 | price=2,100
12 ...
13 [Sim] Done. 22 bids (18 shill).
```

3 Fraud Detection Engine – Complete Source Code

This section presents the four core modules of the real-time fraud detection engine deployed in the AUCTIONHAUS backend. All code is TypeScript, running in-process on the Node.js event loop.

3.1 Type Definitions (`fraud.types.ts`)

Listing 2: Core type interfaces for the fraud engine.

```
1 export interface BidEvent {
2   bidId: string;
3   auctionId: string;
4   bidderId: string;
5   bidderName: string;
6   sellerId: string;
7   auctionTitle: string;
8   amount: number;
9   minIncrement: number;
10  ts: number;           // Unix ms
11  isAutoBid: boolean;
12 }
13
14 export interface FeatureVector {
15   responseTimeMs: number;
16   bidFrequencyPerMin: number;
17   incrementRatio: number;
18   sellerCoOccurrence: number;
```

```

19   reciprocityScore: number;
20 }
21
22 export interface ClassifierWeights {
23   intercept: number;
24   responseTimeMs: number;
25   bidFrequencyPerMin: number;
26   incrementRatio: number;
27   sellerCoOccurrence: number;
28   reciprocityScore: number;
29 }
30
31 export interface NormParams {
32   mean: FeatureVector;
33   std: FeatureVector;
34 }

```

3.2 Sliding-Window Bid Graph (fraud.graph.ts)

Listing 3: In-memory sliding-window bid graph with lazy pruning.

```

1 export const WINDOW_MINUTES = 30;
2 const WINDOW_MS = WINDOW_MINUTES * 60 * 1000;
3
4 export class BidGraph {
5   private auctionBids = new Map<string, BidEvent[]>();
6   private bidderHistory = new Map<string, BidEvent[]>();
7
8   add(event: BidEvent): void {
9     this.prune(event.ts);
10    if (!this.auctionBids.has(event.auctionId))
11      this.auctionBids.set(event.auctionId, []);
12    this.auctionBids.get(event.auctionId)!.push(event);
13    if (!this.bidderHistory.has(event.bidderId))
14      this.bidderHistory.set(event.bidderId, []);
15    this.bidderHistory.get(event.bidderId)!.push(event);
16  }
17
18   bidderSellerCoOccurrence(bidderId: string,
19     sellerId: string): number {
20     const history = this.getBidderHistory(bidderId);
21     const auctionIds = new Set(
22       history.filter(b => b.sellerId === sellerId
23         && b.bidderId !== sellerId)
24         .map(b => b.auctionId)
25     );
26     return auctionIds.size;
27  }
28
29   reciprocityScore(auctionId: string): number {
30     const bids = this.getAuctionBids(auctionId);
31     if (bids.length < 3) return 0;
32     const outbidPairs = new Set<string>();
33     for (let i = 1; i < bids.length; i++) {
34       for (let j = i - 1; j >= 0; j--) {
35         if (bids[j].bidderId !== bids[i].bidderId) {
36           outbidPairs.add(

```

```

37         bids[i].bidderId + "->" + bids[j].bidderId);
38         break;
39     }
40 }
41 }
42 const ids = [...new Set(bids.map(b => b.bidderId))];
43 let total = 0, mutual = 0;
44 for (let i = 0; i < ids.length; i++)
45     for (let j = i + 1; j < ids.length; j++) {
46         total++;
47         if (outbidPairs.has(ids[i]+"->" + ids[j])
48             && outbidPairs.has(ids[j]+"->" + ids[i]))
49             mutual++;
50     }
51 return total === 0 ? 0 : mutual / total;
52 }
53
54 private prune(nowMs: number): void {
55     const cutoff = nowMs - WINDOW_MS;
56     for (const [id, bids] of this.auctionBids) {
57         const t = bids.filter(b => b.ts >= cutoff);
58         if (t.length === 0) this.auctionBids.delete(id);
59         else if (t.length !== bids.length)
60             this.auctionBids.set(id, t);
61     }
62     for (const [id, bids] of this.bidderHistory) {
63         const t = bids.filter(b => b.ts >= cutoff);
64         if (t.length === 0) this.bidderHistory.delete(id);
65         else if (t.length !== bids.length)
66             this.bidderHistory.set(id, t);
67     }
68 }
69 }

```

3.3 Feature Extraction (fraud.features.ts)

Listing 4: Five-feature extraction from the bid graph per event.

```

1 export function extractFeatures(
2     event: BidEvent, graph: BidGraph
3 ): FeatureVector {
4     const bids = graph.getAuctionBids(event.auctionId);
5
6     // F1: Response time (ms since last bid by different bidder)
7     let responseTimeMs = 8000; // neutral default
8     for (let i = bids.length - 1; i >= 0; i--) {
9         if (bids[i].bidderId !== event.bidderId) {
10             responseTimeMs = event.ts - bids[i].ts;
11             break;
12         }
13     }
14
15     // F2: Bid frequency (bids/min across all auctions)
16     const history = graph.getBidderHistory(event.bidderId);
17     const windowStart = event.ts - WINDOW_MINUTES * 60000;
18     const recent = history.filter(b => b.ts >= windowStart);
19     const bidFrequencyPerMin = recent.length / WINDOW_MINUTES;

```

```

20
21 // F3: Increment ratio (amount / minIncrement)
22 let incAmt = event.amount;
23 let hasPrev = false;
24 if (bids.length > 1) {
25     incAmt = event.amount - bids[bids.length - 2].amount;
26     hasPrev = true;
27 }
28 const incrementRatio = event.minIncrement > 0
29     ? (hasPrev ? incAmt / event.minIncrement : 5.0) : 1.0;
30
31 // F4: Seller co-occurrence
32 const sellerCoOccurrence =
33     graph.bidderSellerCoOccurrence(
34         event.bidderId, event.sellerId);
35
36 // F5: Reciprocity
37 const reciprocityScore =
38     graph.reciprocityScore(event.auctionId);
39
40 return { responseTimeMs, bidFrequencyPerMin,
41         incrementRatio, sellerCoOccurrence,
42         reciprocityScore };
43 }

```

3.4 Logistic Regression Classifier (fraud.classifier.ts)

The classifier uses weights learned via gradient descent (Section 4.2). Features are z-scored and clamped to $[-4, +4]$ to prevent outlier domination.

Listing 5: Scoring and explainability functions.

```

1 export const WEIGHTS: ClassifierWeights = {
2     intercept: 2.7426,
3     responseTimeMs: 0.0095,
4     bidFrequencyPerMin: 1.4572,
5     incrementRatio: -0.9484,
6     reciprocityScore: -0.4409,
7     sellerCoOccurrence: 1.7342,
8 };
9
10 export const NORM_PARAMS: NormParams = {
11     mean: { responseTimeMs: 2399.84,
12             bidFrequencyPerMin: 0.1186,
13             incrementRatio: 1.3857,
14             reciprocityScore: 0.2924,
15             sellerCoOccurrence: 0.8714 },
16     std: { responseTimeMs: 1389.50,
17            bidFrequencyPerMin: 0.0678,
18            incrementRatio: 0.8991,
19            reciprocityScore: 0.1785,
20            sellerCoOccurrence: 0.3347 },
21 };
22
23 export const SCORE_THRESHOLD = 0.20;
24
25 function sigmoid(x: number): number {
26     return 1 / (1 + Math.exp(-x));

```



```

27 }
28
29 function normalise(f: FeatureVector,
30                   p: NormParams): FeatureVector {
31   const clamp = (v: number, s: number) =>
32     s > 0 ? Math.max(-4, Math.min(4, v / s)) : 0;
33   return {
34     responseTimeMs: clamp(
35       f.responseTimeMs - p.mean.responseTimeMs,
36       p.std.responseTimeMs),
37     bidFrequencyPerMin: clamp(
38       f.bidFrequencyPerMin - p.mean.bidFrequencyPerMin,
39       p.std.bidFrequencyPerMin),
40     incrementRatio: clamp(
41       f.incrementRatio - p.mean.incrementRatio,
42       p.std.incrementRatio),
43     reciprocityScore: clamp(
44       f.reciprocityScore - p.mean.reciprocityScore,
45       p.std.reciprocityScore),
46     sellerCoOccurrence: clamp(
47       f.sellerCoOccurrence - p.mean.sellerCoOccurrence,
48       p.std.sellerCoOccurrence),
49   };
50 }
51
52 export function score(features: FeatureVector): number {
53   const z = normalise(features, NORM_PARAMS);
54   const logit = WEIGHTS.intercept
55     + WEIGHTS.responseTimeMs * z.responseTimeMs
56     + WEIGHTS.bidFrequencyPerMin * z.bidFrequencyPerMin
57     + WEIGHTS.incrementRatio * z.incrementRatio
58     + WEIGHTS.reciprocityScore * z.reciprocityScore
59     + WEIGHTS.sellerCoOccurrence * z.sellerCoOccurrence;
60   return sigmoid(logit);
61 }

```

3.5 Engine Orchestrator (fraud.engine.ts)

The FraudEngine singleton is invoked after every committed bid. It wires the graph, feature extraction, and classifier into a single call path and emits fraud:flag events over Socket.io to the admin dashboard.

Listing 6: Singleton fraud engine orchestrating the detection pipeline.

```

1 export class FraudEngine {
2   private static instance: FraudEngine;
3   private graph = new BidGraph();
4   private io: Server | null = null;
5
6   static getInstance(): FraudEngine {
7     if (!FraudEngine.instance)
8       FraudEngine.instance = new FraudEngine();
9     return FraudEngine.instance;
10  }
11
12  init(io: Server): void { this.io = io; }
13 }

```

```

14  async onBid(event: BidEvent): Promise<void> {
15      this.graph.add(event);
16      const features = extractFeatures(event, this.graph);
17      const fraudScore = score(features);
18      if (fraudScore < SCORE_THRESHOLD) return;
19
20      const flag: FraudFlagEvent = {
21          id: uuidv4(), ts: event.ts,
22          bidId: event.bidId, bidderId: event.bidderId,
23          bidderName: event.bidderName,
24          auctionId: event.auctionId,
25          auctionTitle: event.auctionTitle,
26          amount: event.amount,
27          score: fraudScore, features,
28          reason: explain(features, fraudScore),
29      };
30
31      // Non-blocking emit to admin room
32      if (this.io)
33          this.io.to('admin:fraud').emit('fraud:flag', flag);
34
35      // Fire-and-forget DB persistence
36      void prisma.fraudFlag.create({ data: {
37          id: flag.id, bidderId: flag.bidderId,
38          auctionId: flag.auctionId, bidId: flag.bidId,
39          score: flag.score,
40          features: flag.features as object,
41          reason: flag.reason,
42      }}).catch(err =>
43          console.warn('FraudEngine persist fail:', err.message)
44      );
45  }
46  }

```

4 Dataset Generation and Training Pipeline

4.1 Dataset Construction

The training corpus is constructed by executing the simulation loop (Section 2) multiple times, each producing an `events.jsonl` file. The training script (`packages/simulator/src/train.ts`) reads all runs from `packages/simulator/runs/`, replays each bid event through the production `BidGraph` and `extractFeatures` pipeline, and assembles a labelled feature matrix.

Why a synthetic dataset? No publicly available dataset provides ground-truth skill labels for live English auctions with sub-second timing resolution. Existing corpora (e.g., eBay post-auction dumps) lack real-time timestamps and contain only post-hoc annotations. Our simulator generates data with deterministic labels and full timing fidelity, making every experiment fully reproducible.

The trainer output below is the verbatim console log:

Listing 7: Actual `train.ts` output.

```

1 [Trainer] Found 3 simulation run directories.
2 [Trainer] Extracted 70 training examples
3           (59 skill / 11 truthful).

```

Table 2: Training Corpus Statistics (from `train.ts` output)

Statistic	Value
Total bid events (N)	70
Fraudulent bids (shill + collusion)	59 (84.3%)
Non-fraudulent bids (truthful + sniper)	11 (15.7%)
Simulation runs aggregated	3
Auction format	English (ascending)
Features per sample	5
Ground-truth source	Simulator labels (<code>isShill</code>)

4.2 Feature Scaling: Z-Score Normalisation

Because our five features operate on different scales (milliseconds, bids/minute, dimensionless ratios, counts, and proportions), every feature is standardised before classification:

$$z_i = \frac{x_i - \mu_i}{\sigma_i} \quad (1)$$

Table 3 lists the empirically computed normalisation constants derived from the training corpus by the `train.ts` pipeline.

Table 3: Empirical Feature Normalisation Constants

Symbol	Feature	Mean (μ)	Std (σ)
τ	Response Latency	2399.84 ms	1389.50 ms
ϕ	Bid Frequency	0.1186 bids/min	0.0678 bids/min
ρ	Increment Ratio	1.3857	0.8991
σ	Seller Co-occurrence	0.8714	0.3347
γ	Outbid Reciprocity	0.2924	0.1785

Additionally, each normalised feature is clamped to the range $[-4, +4]$ to prevent extreme outliers from dominating the logistic function.

4.3 Gradient Descent Training

The logistic regression model is trained via batch gradient descent minimising the binary cross-entropy loss with L2 regularisation:

$$\mathcal{L}(\mathbf{w}, b) = -\frac{1}{N} \sum_{i=1}^N [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)] + \frac{\lambda}{2} \|\mathbf{w}\|^2 \quad (2)$$

where $\hat{y}_i = \sigma(b + \mathbf{w}^\top \mathbf{z}_i)$.

4.4 Learned Model Parameters

Table 5 presents the weight vector after convergence.

Table 4: Training Hyperparameters

Hyperparameter	Value
Learning rate (α)	0.20
L2 regularisation (λ)	0.01
Epochs	10,000
Weight initialisation	All zeros
Gradient computation	Full batch

Table 5: Learned Logistic Regression Weights

Parameter	Value	Interpretation
Intercept (β_0)	+2.7426	Positive bias towards fraud class
Response time (β_1)	+0.0095	Near-zero: weak discriminator
Bid frequency (β_2)	+1.4572	High frequency increases fraud score
Increment ratio (β_3)	-0.9484	Low ratio (min-increment) is suspicious
Seller co-occurrence (β_4)	+1.7342	Strongest weight: dominant feature
Reciprocity (β_5)	-0.4409	Moderate negative (z-score inverted)

The large positive weight on seller co-occurrence ($\beta_4 = +1.7342$) confirms the ablation finding in the main paper: this feature carries the most discriminative power. The positive intercept ($\beta_0 = +2.7426$) reflects the model’s recall-optimised prior, which is appropriate for the asymmetric cost structure where missed fraud is far more costly than a false alarm sent to an administrator for manual review.

The training script reports the following performance on the training set (verbatim from the console output):

Listing 8: Training performance (verbatim from train.ts).

```

1 [Trainer] Training Performance:
2   Accuracy:  97.14%
3   Precision: 0.9672
4   Recall:    1.0000
5   F1-Score:  0.9833

```

4.5 Training Convergence

The training pipeline logs loss values at regular intervals. The following values are taken directly from the `train.ts` console output:

The loss converges rapidly within the first 2,000 epochs and stabilises at 0.1165, indicating that the model has reached its optimum given the feature space and regularisation strength. The early plateau is expected for a well-separated binary classification task with only five features.

Table 6: Training Loss Convergence (verbatim from `train.ts`)

Epoch	Loss
1	0.693187
2,000	0.116521
4,000	0.116521
6,000	0.116521
8,000	0.116521
10,000	0.116521

5 Extended Experimental Results

5.1 Full ROC Sweep

Table 7 reports the classifier performance at every threshold θ from 0.05 to 0.95 in steps of 0.05. These values are computed by the evaluation harness (`packages/simulator/src/eval.ts`) replaying the simulation events through the production classifier.

Table 7: Full ROC Sweep: Per-Threshold Performance Metrics

θ	TP	FP	TN	FN	F1	FPR
0.05	18	13	39	0	0.973	0.250
0.10	18	13	39	0	0.973	0.250
0.15	18	13	39	0	0.973	0.250
0.20	18	0	52	0	1.000	0.000
0.25	18	0	52	0	1.000	0.000
0.30	18	0	52	0	1.000	0.000
0.35	18	0	52	0	1.000	0.000
0.40	18	0	52	0	1.000	0.000
0.45	18	0	52	0	1.000	0.000
0.50	18	0	52	0	1.000	0.000
0.55	18	0	52	0	1.000	0.000
0.60	18	0	52	0	1.000	0.000
0.65	18	0	52	0	1.000	0.000
0.70	18	0	52	0	1.000	0.000
0.75	18	0	52	0	1.000	0.000
0.80	18	0	52	0	1.000	0.000
0.85	18	0	52	0	1.000	0.000
0.90	18	0	52	0	1.000	0.000
0.95	10	0	52	8	0.714	0.000

The classifier maintains perfect precision and recall across the wide operating range $\theta \in [0.20, 0.90]$, indicating that the learned decision boundary produces well-separated score distributions for the two classes. The sharp transition at $\theta = 0.20$ (where 13 false positives disappear) identifies this as the optimal operating point for recall-sensitive deployments.

5.2 Detailed Feature Ablation

Table 8 extends the ablation study from the main paper, reporting the full confusion matrix for each zeroed-out feature.

Table 8: Feature Ablation: Detailed Metrics at $\theta = 0.20$

Feature Removed	P	R	F1	$\Delta F1$
None (full model)	1.000	1.000	1.000	–
Response time (τ)	1.000	1.000	1.000	0.000
Bid frequency (ϕ)	1.000	0.944	0.971	−0.029
Increment ratio (ρ)	1.000	1.000	1.000	0.000
Seller co-occ. (σ)	1.000	0.056	0.105	−0.895
Reciprocity (γ)	1.000	1.000	1.000	0.000

The $\Delta F1$ column quantifies each feature’s marginal contribution. Seller co-occurrence accounts for 89.5 percentage points of F1 performance. Bid frequency contributes a smaller but non-trivial 2.9 points. The remaining three features (response time, increment ratio, reciprocity) are individually redundant given the presence of seller co-occurrence, but they provide defence-in-depth against adversaries who might vary their seller-targeting patterns.

5.3 Baseline Comparison

The outbid-count baseline (flag a bidder if they have been outbid more than 10 times on the same auction) produces $F1 = 0.000$ on our corpus because no bidder reaches the threshold of 10 outbids within the 60-second simulation window. This confirms that simple heuristics are insufficient for short-duration auctions and motivates the graph-based streaming approach.

5.4 Throughput Benchmarks: Extended k6 Results

Table 9 presents the p95 latency measurements from our k6 load-test harness, measured on a single developer workstation. Each row represents a separate k6 run with the specified number of virtual users (VUs) and a fixed 15-second duration. The p95 values below are taken directly from the k6 summary output.

Eval output (verbatim from eval.ts):

Listing 9: Actual eval.ts console output.

```

1 [Eval] 22 bid events, 18 shill
2 [Eval] Best threshold: 0.20 | P=1.000 R=1.000 F1=1.000
3 [Eval] Ablation:
4   responseTimeMs      F1=1.000 P=1.000 R=1.000
5   bidFrequencyPerMin  F1=0.971 P=1.000 R=0.944
6   incrementRatio      F1=1.000 P=1.000 R=1.000
7   sellerCoOccurrence  F1=0.105 P=1.000 R=0.056
8   reciprocityScore    F1=1.000 P=1.000 R=1.000
9 [Eval] Baseline F1=0.000 -> LR F1=1.000 (+100.0%)

```

At 100 concurrent users, the Redis Stream path achieves a p95 of just 8.1 ms compared to 5,771.7 ms for the FOR UPDATE path – a **32.7×** improvement in throughput (975 itr/s vs 30 itr/s). The stream path’s latency *decreases* as concurrency rises because the consumer group batches queued events.

Table 9: k6 Bid Throughput: p95 Latency (from actual k6 runs)

Mode	VUs	Duration	p95	Iterations	itr/s
FOR UPDATE	1	15 s	236.0 ms	53	3.5
FOR UPDATE	10	15 s	1,284.4 ms	143	9.0
FOR UPDATE	100	15 s	5,771.7 ms	570	29.8
Redis Stream	1	15 s	133.9 ms	86	5.7
Redis Stream	10	15 s	47.9 ms	1,392	92.1
Redis Stream	100	15 s	8.1 ms	14,626	975.1

6 Formal Proofs and Complexity Analysis

6.1 Commit-Reveal Protocol: Security Properties

Definition 1 (Commitment Scheme). A commitment scheme $\mathcal{C} = (\text{Commit}, \text{Reveal})$ over message space $\mathcal{M}$ satisfies two properties:

- **Hiding:** Given $c = \text{Commit}(m, r)$, no probabilistic polynomial-time adversary $\mathcal{A}$ can determine m with probability non-negligibly better than $1/|\mathcal{M}|$.
- **Binding:** No probabilistic polynomial-time adversary $\mathcal{A}$ can find $(m, r) \neq (m', r')$ such that $\text{Commit}(m, r) = \text{Commit}(m', r')$ with non-negligible probability.

Theorem 1 (SHA-256 Commitment Security). The commitment function $H = \text{SHA-256}(\text{amountHex} \parallel \text{nonce})$, where nonce is a uniformly random 256-bit string, satisfies both hiding and binding under the random oracle model.

Proof. Hiding. In the random oracle model, SHA-256 outputs are uniformly distributed over $\{0, 1\}^{256}$ for distinct inputs. Since the nonce is 256 bits of uniform randomness and is concatenated with the amount, the input to the hash is unique with overwhelming probability $(1 - 2^{-256})$. An adversary observing only H gains no information about the amount prefix, as the oracle output is independent of any proper substring of its input.

Binding. Suppose an adversary finds $(a, n) \neq (a', n')$ with $H(a \parallel \text{nonce} \parallel n) = H(a' \parallel \text{nonce} \parallel n')$. This constitutes a collision in SHA-256, which contradicts the collision-resistance assumption. For SHA-256, the best known collision attack requires $O(2^{128})$ operations, which is computationally infeasible. $\square$

Proposition 2 (Limitation: No Range Proofs). The SHA-256 commitment scheme does not support server-side verification of $\text{amount} \geq \text{reservePrice}$ without revealing the amount. Achieving this requires Pedersen commitments with Bulletproof range proofs, which add $O(\log n)$ proof size and $O(n)$ verification cost per bid.

6.2 Graph Algorithm Complexity

Theorem 3 (Feature Extraction Complexity). For a bid graph $\mathcal{G}$ with n total bid events in the sliding window, feature extraction for one incoming bid event runs in $O(n)$ worst-case time and $O(n)$ space.

Proof. We analyse each feature independently:

F1 (Response time): Scans the auction bid list backwards until a different bidder is found. Worst case: $O(n_a)$ where n_a is the number of bids on this auction.

F2 (Bid frequency): Reads the bidder history array and filters by timestamp. Cost: $O(n_b)$ where n_b is the bidder’s history length.

F3 (Increment ratio): Constant-time lookup of the penultimate bid in the auction list. Cost: $O(1)$.

F4 (Seller co-occurrence): Scans the bidder’s history, filters by seller ID, and counts distinct auction IDs via a hash set. Cost: $O(n_b)$.

F5 (Reciprocity): For k distinct bidders on the auction, constructs the outbid-pairs set in $O(n_a)$ and checks mutual pairs in $O(k^2)$. Since $k \leq n_a$, this is $O(n_a^2)$ in the worst case.

Total: $O(n_a^2 + n_b)$. Since $n_a, n_b \leq n$, the overall complexity is $O(n^2)$ worst-case. However, in practice $n_a \ll n$ (bids per auction within the 30-minute window are typically < 100), making the amortised cost effectively $O(1)$ per bid. Our benchmarks confirm sub-millisecond execution. $\square$

Theorem 4 (Sliding Window Space Bound). *The bid graph uses $O(B \cdot W)$ memory, where B is the peak bid arrival rate (bids/second) and W is the window width in seconds.*

Proof. Each bid event is stored in two maps (auction and bidder). The pruning step removes all events older than W seconds on every insertion. At steady state, the graph contains at most $B \cdot W$ events in each map. With $W = 1800$ s and a peak rate of $B = 800$ bids/s (from our Redis Stream benchmark), the upper bound is approximately 1.44×10^6 events, each consuming ~ 200 bytes, for a total of ~ 288 MB. $\square$

6.3 Gradient Descent Convergence

Proposition 5 (Convergence of L2-Regularised Logistic Regression). *The loss function $\mathcal{L}(\mathbf{w}, b)$ is strictly convex when $\lambda > 0$. Full-batch gradient descent with a sufficiently small learning rate α converges to the unique global minimum.*

Proof. The binary cross-entropy loss is convex in $\mathbf{w}$ and b (the Hessian is positive semidefinite). Adding the L2 penalty $\frac{\lambda}{2} \|\mathbf{w}\|^2$ with $\lambda > 0$ makes the Hessian positive definite, guaranteeing strict convexity and a unique minimiser. For a β -smooth, λ -strongly convex function, gradient descent with $\alpha \leq 1/\beta$ converges at rate $O((1 - \lambda/\beta)^t)$. Our empirical loss curve (Table 6) confirms monotonic decrease to a stable minimum. $\square$

7 k6 Load Test Harness

The throughput benchmarks in the main paper (Table III) and the extended results (Table 9) were generated using the following k6 load-test script. The script supports two modes: `direct` (standard SELECT ... FOR UPDATE path) and `stream` (Redis Stream sequencer path), selectable via the `MODE` environment variable.

Listing 10: k6 bid throughput benchmark script.

```
1 import http from 'k6/http';
2 import { check, sleep } from 'k6';
3 import { Trend, Counter } from 'k6/metrics';
4
5 const bidLatency = new Trend('bid_latency', true);
6 const bidErrors = new Counter('bid_errors');
```



```

8 export const options = {
9   scenarios: { constant_load: {
10     executor: 'constant-vus',
11     vus: __ENV.VUS ? parseInt(__ENV.VUS) : 10,
12     duration: __ENV.DURATION || '30s',
13   }},
14   thresholds: {
15     'bid_latency{mode:direct}': ['p(99)<2000'],
16     'bid_latency{mode:stream}': ['p(99)<500'],
17     'bid_errors': ['count<5'],
18   },
19 };
20
21 const BASE = __ENV.BASE_URL
22   || 'http://localhost:5000/api';
23 const TOKEN = __ENV.SIM_TOKEN || '';
24 const AUCTION_ID = __ENV.AUCTION_ID || '';
25 const headers = {
26   'Content-Type': 'application/json',
27   'Authorization': 'Bearer ${TOKEN}',
28 };
29
30 export default function () {
31   const amount = 1000 + (__ITER * 100) + (__VU * 10);
32   const mode = __ENV.MODE === 'stream'
33     ? 'stream' : 'direct';
34   const url = mode === 'stream'
35     ? `${BASE}/bids/auctions/${AUCTION_ID}/stream`
36     : `${BASE}/bids/auctions/${AUCTION_ID}`;
37
38   const res = http.post(url,
39     JSON.stringify({ amount }),
40     { headers, tags: { mode } });
41   bidLatency.add(res.timings.duration, { mode });
42
43   check(res, {
44     'bid accepted': (r) =>
45       [200, 201, 202, 400].includes(r.status),
46   });
47   sleep(0.1);
48 }

```

Invocation:

```

1 # Direct mode at 100 VUs
2 k6 run packages/simulator/k6/bid-throughput.js \
3   -e MODE=direct -e VUS=100 -e DURATION=15s \
4   -e SIM_TOKEN=<jwt> -e AUCTION_ID=<uuid>
5
6 # Redis Stream mode at 100 VUs
7 k6 run packages/simulator/k6/bid-throughput.js \
8   -e MODE=stream -e VUS=100 -e DURATION=15s \
9   -e SIM_TOKEN=<jwt> -e AUCTION_ID=<uuid>

```

8 Sealed-Bid Commitment: Implementation

The commit-reveal protocol described in the main paper (Section VII) is implemented as a standalone service module. The key functions are presented below.

Listing 11: SHA-256 commitment hashing and verification.

```
1 import { createHash } from 'crypto';
2
3 // Server-side hash computation
4 export function hashCommitment(
5   amountCents: number, nonce: string
6 ): string {
7   const amountHex = amountCents.toString(16);
8   return createHash('sha256')
9     .update(`${amountHex}:${nonce}`)
10    .digest('hex');
11 }
12
13 // Commit phase: store hash, never see amount
14 export const commitBid = async (data: {
15   auctionId: string;
16   bidderId: string;
17   commitHash: string;
18 }) => {
19   const auction = await prisma.auction.findUnique({
20     where: { id: data.auctionId }
21   });
22   if (auction.type !== 'SEALED_BID')
23     throw createError('Not a sealed-bid auction', 400);
24   if (auction.status !== 'ACTIVE')
25     throw createError('Auction is not active', 400);
26
27   // Upsert: bidder may update commitment
28   return prisma.bidCommitment.upsert({
29     where: { auctionId_bidderId: {
30       auctionId: data.auctionId,
31       bidderId: data.bidderId
32     }},
33     update: { commitHash: data.commitHash,
34               revealedAt: null,
35               revealedAmount: null },
36     create: { ...data },
37   });
38 };
39
40 // Reveal phase: verify hash match
41 export const revealBid = async (data: {
42   auctionId: string;
43   bidderId: string;
44   amount: number;
45   nonce: string;
46 }) => {
47   const commitment = await prisma.bidCommitment
48     .findUnique({ where: {
49       auctionId_bidderId: {
50         auctionId: data.auctionId,
51         bidderId: data.bidderId
52       }
53     }
54   });
```

```

53     });
54
55     const computed = hashCommitment(
56       Math.round(data.amount * 100), data.nonce);
57     const isValid = computed === commitment.commitHash;
58
59     await prisma.bidCommitment.update({
60       where: { id: commitment.id },
61       data: {
62         revealedAt: new Date(),
63         revealedAmount: isValid ? data.amount : null,
64         isValid,
65       },
66     });
67
68     if (!isValid) throw createError(
69       'Reveal failed: hash mismatch', 400);
70     return { valid: true, amount: data.amount };
71   };

```

9 Reproducibility Instructions

We believe that reproducibility is essential for honest research. This section provides step-by-step instructions so that any reviewer or researcher can independently verify every claim in the paper.

9.1 Monorepo Structure

AuctionHaus is organised as an npm workspaces monorepo. The root `package.json` defines three workspaces:

Listing 12: Root `package.json` (abbreviated).

```

1 {
2   "name": "auctionhaus-monorepo",
3   "workspaces": ["front", "back", "packages/simulator"],
4   "scripts": {
5     "dev:back": "npm run dev --workspace=back",
6     "sim:run": "npm run sim:run --workspace=back",
7     "train:fraud": "npm run train:fraud --workspace=back",
8     "eval:fraud": "npm run eval:fraud --workspace=back",
9     "prisma:migrate": "npx prisma migrate dev ..."
10  }
11 }

```

All simulator, training, and evaluation commands are run from the **repository root** using `npm run <script>`. There is no need to `cd` into subdirectories.

9.2 Prerequisites

1. **Node.js** ≥ 18 and **npm** ≥ 9
2. **PostgreSQL 15** – local installation or Docker
3. **Redis 7** – we use a lightweight Alpine container

4. **k6** – install from <https://k6.io/docs/getting-started/installation/>
5. **A LaTeX distribution** – for compiling the paper (optional)

9.3 Step-by-Step Setup

Step 1: Clone and install dependencies.

```
1 git clone https://github.com/prathamreet/auctionhaus
2 cd auctionhaus
3 npm install
```

The `postinstall` hook automatically runs `prisma generate` to create the database client.

Step 2: Configure environment variables.

Copy the provided example file and adjust if needed:

```
1 cp back/.env.example back/.env
```

The `back/.env.example` contains all required variables:

Listing 13: `back/.env.example` contents.

```
1 PORT=5000
2 NODE_ENV=development
3 DATABASE_URL="postgresql://postgres:password
4   @localhost:5432/auctionhaus"
5 JWT_SECRET=your_super_secret_jwt_key
6 JWT_EXPIRES_IN=7d
7 REDIS_URL=redis://localhost:6379
8 FRONTEND_URL=http://localhost:3000
```

Make sure the `DATABASE_URL` points to a running PostgreSQL instance and the `REDIS_URL` points to a running Redis instance.

Step 3: Start Redis.

We do not use Docker Compose. Start a standalone Redis container:

```
1 docker run --name auctionhaus-redis \
2   -p 6379:6379 -d redis:alpine
```

Step 4: Start PostgreSQL.

If you do not have PostgreSQL installed locally, use Docker:

```
1 docker run --name auctionhaus-pg \
2   -e POSTGRES_PASSWORD=password \
3   -e POSTGRES_DB=auctionhaus \
4   -p 5432:5432 -d postgres:15-alpine
```

Step 5: Run database migrations.

```
1 npm run prisma:migrate
```

Step 6: Start the backend server.

```
1 npm run dev:back
```

The server starts on `http://localhost:5000`. Keep this terminal open.

9.4 Running the Experiments

Step 7: Run the simulator (generates ground-truth bid events).

```
1 npm run sim:run
```

Output is saved to `packages/simulator/runs/<runId>/`.

Step 8: Train the classifier (learns weights from all runs).

```
1 npm run train:fraud
```

This reads all runs, extracts features, trains the logistic regression model, and writes the learned weights directly into `back/src/modules/fraud/fraud.classifier.ts`.

Step 9: Evaluate the classifier (generates ROC, ablation, and LaTeX tables).

```
1 npm run eval:fraud
```

This outputs `paper/figures/roc_data.json`, `paper/figures/ablation_data.json`, and `paper/tables/metrics`.

9.5 Running k6 Throughput Benchmarks

The k6 benchmark requires a valid JWT token and an active auction ID. Here is how to obtain them:

Step 10a: Create a benchmark user.

Register a user through the frontend (`http://localhost:3000`) or via the API:

```
1 curl -X POST http://localhost:5000/api/auth/register \
2   -H "Content-Type: application/json" \
3   -d '{"name":"Bench User",
4       "email":"bench@auctionhaus.com",
5       "password":"BenchPass123!"}'
```

Step 10b: Log in to obtain a JWT token.

```
1 curl -X POST http://localhost:5000/api/auth/login \
2   -H "Content-Type: application/json" \
3   -d '{"email":"bench@auctionhaus.com",
4       "password":"BenchPass123!"}'
```

The response includes a token field. Copy it.

Step 10c: Create a test auction.

Create an auction through the frontend (logged in as a different user) or note the auction ID from a simulator run. The auction must be in ACTIVE status.

Step 10d: Run k6.

```
1 # Direct path (FOR UPDATE)
2 k6 run packages/simulator/k6/bid-throughput.js \
3   -e SIM_TOKEN="<your-jwt-token>" \
4   -e AUCTION_ID="<auction-uuid>" \
5   -e VUS=100 -e DURATION="15s" \
6   -e MODE="direct"
7
8 # Redis Stream path
9 k6 run packages/simulator/k6/bid-throughput.js \
10  -e SIM_TOKEN="<your-jwt-token>" \
11  -e AUCTION_ID="<auction-uuid>" \
12  -e VUS=100 -e DURATION="15s" \
13  -e MODE="stream"
```

Run at VUs = 1, 10, and 100 for both modes to reproduce Table 9.

9.6 Output Artefacts

Table 10: Generated Output Files

File	Description
packages/simulator/runs/<id>/events.jsonl	Ground-truth labelled bid log
packages/simulator/runs/<id>/manifest.json	Run metadata and agent map
paper/figures/roc_data.json	ROC curve coordinates
paper/figures/ablation_data.json	Per-feature ablation metrics
paper/tables/metrics.tex	Auto-generated LaTeX table
back/src/modules/fraud/fraud.classifier.ts	Updated model weights

9.7 Verification Checklist

To verify our claims, check the following after running the pipeline:

1. **Classifier accuracy:** The `train:fraud` output should report $\text{Accuracy} \geq 97\%$, $\text{F1} \geq 0.98$.
2. **Best threshold:** The `eval:fraud` output should report $\text{best threshold} = 0.20$ with $\text{P} = 1.000$, $\text{R} = 1.000$, $\text{F1} = 1.000$.
3. **Ablation dominance:** Removing `sellerCoOccurrence` should drop F1 to ≈ 0.105 ; other features should have minimal impact.
4. **Baseline:** The `outbid-count` baseline should report $\text{F1} = 0.000$.
5. **Throughput:** At $\text{VUs} = 100$, `Redis Stream p95` should be $< 50 \text{ ms}$ while `FOR UPDATE p95` should be $> 1,000 \text{ ms}$.